

Java Backend Architecture

Interview Series

Chapter 7.5

Gradle Build Script & Tasks

50+ Interview Q&A

Code Examples

Architecture Diagrams

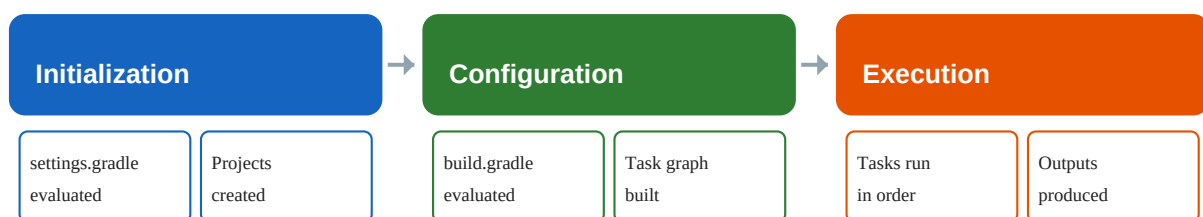
Advanced Topics

Chapter 7.5 — Gradle Build Script & Tasks

Introduction

Gradle is a build automation tool using Groovy or Kotlin DSL (Domain Specific Language) instead of XML. It offers a three-phase build lifecycle (Initialization, Configuration, Execution), a rich task model with up-to-date checking and lazy configuration, and significantly better performance than Maven through incremental builds and build caching. Spring Boot, Android, and many major open-source projects use Gradle as their primary build tool.

Gradle Build Phases



Every build.gradle runs during Configuration — even if the task is not executed. Use lazy configuration to avoid this.

Phase	What Happens	Files Involved
Initialization	Evaluates settings.gradle(/.kts) — determines which projects participate in the build	settings.gradle
Configuration	Evaluates all build.gradle(/.kts) files — registers tasks, plugins, and dependencies	build.gradle, build.gradle.kts, plugins, dependencies
Execution	Runs only the requested tasks and their dependencies	Task inputs/outputs

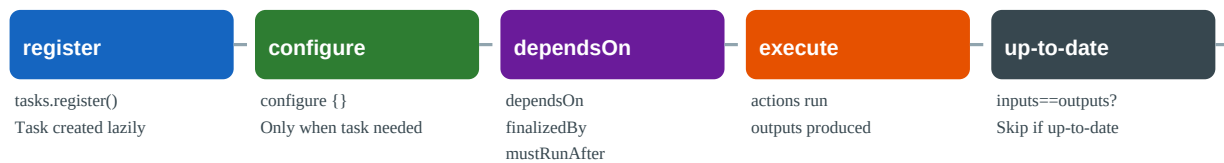
Groovy DSL vs Kotlin DSL

Aspect	Groovy DSL (build.gradle)	Kotlin DSL (build.gradle.kts)
Syntax	Dynamic, concise	Statically typed, verbose
IDE support	Limited autocomplete	Full IDE support (IntelliJ)
Refactoring	Difficult	IDE-assisted refactoring
Performance	Slightly faster config	Slightly slower first config
Adoption	Legacy / existing projects	New projects (recommended)
Strings	Single or double quotes	Double quotes only

Kotlin DSL build.gradle.kts

```
// settings.gradle.kts rootProject.name = "my-app" include("domain-model", "service-layer",
"web-api") // build.gradle.kts (root) plugins { id("org.springframework.boot") version "3.2.0"
apply false id("io.spring.dependency-management") version "1.1.4" apply false java }
subprojects { apply(plugin = "java") apply(plugin = "io.spring.dependency-management")
repositories { mavenCentral() } java { toolchain { languageVersion = JavaLanguageVersion.of(17)
} } tasks.withType<Test> { useJUnitPlatform() maxParallelForks =
Runtime.getRuntime().availableProcessors() } }
```

Task Lifecycle



tasks.register vs tasks.create

```
// tasks.register – LAZY (preferred): task only configured when needed
tasks.register<Copy>("copyDocs") { from("src/docs") into(layout.buildDirectory.dir("docs")) }
// tasks.create – EAGER (avoid): task configured on every build tasks.create("eagerTask") {
doLast { println("This always configures!") } } // Named task types for common operations:
tasks.register<Copy>("copyConfig") { from("src/main/config")
into(layout.buildDirectory.dir("config")) include("*.properties", "*.yaml") filter { line ->
line.replace("localhost", "prod-server") } } tasks.register<Zip>("archiveDocs") {
from(tasks.named("copyDocs")) archiveFileName.set("docs-${version}.zip")
destinationDirectory.set(layout.buildDirectory.dir("archives")) }
```

Task Dependencies

```
// dependsOn: this task requires another to run first tasks.named("test") {
dependsOn(tasks.named("generateTestData")) } // finalizedBy: always run another task after this
one (even on failure) tasks.named("integrationTest") {
finalizedBy(tasks.named("stopTestContainers")) } // mustRunAfter: ordering without dependency
(both must be explicitly requested) tasks.named("test") {
mustRunAfter(tasks.named("unitTest")) } // shouldRunAfter: soft ordering hint
tasks.named("check") { shouldRunAfter(tasks.named("test")) }
```

Up-to-Date Checking & Incremental Builds

Gradle skips task execution if inputs and outputs haven't changed. Declare inputs and outputs explicitly for custom tasks to enable this optimisation. A task is UP-TO-DATE if: all input files unchanged, all input properties unchanged, all output files exist and are unchanged.

```
abstract class GenerateReport : DefaultTask() { @get:InputDirectory abstract val inputDir:
DirectoryProperty @get:InputProperty abstract val reportTitle: Property<String>
@get:OutputFile abstract val outputFile: RegularFileProperty @TaskAction fun generate() { //
Gradle tracks inputs/outputs for up-to-date check val content = inputDir.get().asFile .walk()
.filter { it.isFile } .joinToString(" ") { it.name } outputFile.get().asFile.writeText("#
${reportTitle.get()} $content") } } tasks.register<GenerateReport>("generateReport") {
inputDir.set(layout.projectDirectory.dir("src/main/resources")) reportTitle.set("Build
Report") outputFile.set(layout.buildDirectory.file("reports/report.md")) }
```

Gradle Wrapper (gradlew)

```
# Generate the Gradle Wrapper gradle wrapper --gradle-version=8.5 # This creates: # gradlew
(Unix executable) # gradlew.bat (Windows executable) #
gradle/wrapper/gradle-wrapper.properties # gradle/wrapper/gradle-wrapper.jar #
gradle-wrapper.properties
distributionUrl=https://services.gradle.org/distributions/gradle-8.5-bin.zip
zipStoreBase=GRADLE_USER_HOME zipStorePath=wrapper/dists # Use wrapper (recommended in all
environments) ./gradlew clean build ./gradlew test --tests "com.example.*ServiceTest"
```

Common Gradle Tasks

Task	Description
<code>./gradlew build</code>	Compile, test, assemble (full build)
<code>./gradlew clean</code>	Delete build/ directory
<code>./gradlew test</code>	Run unit tests
<code>./gradlew check</code>	Run all checks (test + static analysis)
<code>./gradlew assemble</code>	Assemble outputs without running tests
<code>./gradlew dependencies</code>	Print dependency tree
<code>./gradlew dependencyInsight --dependency slf4j</code>	Show why a dependency was included
<code>./gradlew tasks</code>	List available tasks
<code>./gradlew tasks --all</code>	List all tasks including sub-tasks
<code>./gradlew :module:taskName</code>	Run task in specific subproject
<code>./gradlew --scan</code>	Upload build scan to scans.gradle.com
<code>./gradlew --profile</code>	Generate HTML performance report
<code>./gradlew --dry-run</code>	Show what would run without running it
<code>./gradlew --continue</code>	Continue after task failure

50+ Interview Questions & Answers

Q1. What are Gradle's three build phases?

Initialization (evaluates `settings.gradle`, determines project structure), Configuration (evaluates all build scripts, builds task graph), Execution (runs requested tasks in order).

Q2. What is the difference between Initialization and Configuration phases?

Initialization reads `settings.gradle` and creates Project objects. Configuration evaluates each `build.gradle` to configure tasks and plugins. Both happen BEFORE any task executes.

Q3. Why is understanding the Configuration phase important?

All `build.gradle` code runs during Configuration — even if a task never executes. Expensive operations in build scripts slow every build. Use lazy configuration (providers, `tasks.register`) to defer work to Execution.

Q4. What is the difference between Groovy DSL and Kotlin DSL?

Groovy DSL (`build.gradle`) uses dynamic Groovy — concise but limited IDE support. Kotlin DSL (`build.gradle.kts`) is statically typed — full IDE autocomplete, refactoring, and compile-time error detection. Kotlin DSL is recommended for new projects.

Q5. What is `tasks.register` vs `tasks.create`?

`register` is lazy — the task is only configured when it's actually needed (in the task graph). `create` is eager — the task is configured during Configuration phase regardless. Always prefer `register` for performance.

Q6. What is the Gradle task graph?

A directed acyclic graph (DAG) of all tasks and their dependencies, built during Configuration phase. Only tasks reachable from the requested tasks are executed.

Q7. What does dependsOn mean for a Gradle task?

The task cannot start until all dependsOn tasks complete successfully. If a dependency fails, the task is skipped.

Q8. What does finalizedBy mean?

The specified task always runs after this task, even if this task fails. Used for cleanup: stopTestContainers always runs after integrationTest.

Q9. What does mustRunAfter mean?

Ordering constraint without dependency — if both tasks are in the execution plan, this task runs after the specified one. But it doesn't force the other task to execute.

Q10. What is an up-to-date task?

A task whose inputs and outputs haven't changed since the last execution. Gradle skips UP-TO-DATE tasks and reports them as such in the build output.

Q11. How does Gradle determine if a task is up-to-date?

Compares hash of all @InputFiles, @InputDirectories, @Input properties with the stored snapshot. If all match and all @OutputFiles exist with matching hashes — the task is UP-TO-DATE.

Q12. What annotations declare task inputs/outputs?

@InputFile, @InputFiles, @InputDirectory, @InputDirectories, @Input (property), @OutputFile, @OutputFiles, @OutputDirectory, @OutputDirectories. These enable up-to-date checking and build cache.

Q13. What is the Gradle Wrapper?

Shell scripts (gradlew/gradlew.bat) that download and use a specific Gradle version defined in gradle-wrapper.properties. Ensures build reproducibility without requiring Gradle installed on the system.

Q14. What file defines the Gradle version in the wrapper?

gradle/wrapper/gradle-wrapper.properties — contains distributionUrl pointing to the specific Gradle distribution zip.

Q15. How do you upgrade the Gradle Wrapper version?

./gradlew wrapper --gradle-version=8.5 — updates gradle-wrapper.properties to the new version. Commit the updated properties file to source control.

Q16. What is settings.gradle.kts used for?

Defines the root project name and declares which subprojects are included in the build. Must be present for multi-project builds. Also configures pluginManagement and dependencyResolutionManagement.

Q17. What is the purpose of allprojects vs subprojects in build.gradle?

allprojects applies configuration to all projects including the root. subprojects applies only to child projects (not the root). Use subprojects for applying the java plugin and dependencies.

Q18. What is a Task type in Gradle?

A reusable task class like Copy, Zip, JavaExec, Test, etc. Instead of defining all behaviour inline, tasks.register creates a typed task with pre-defined inputs/outputs and actions.

Q19. What is the doFirst and doLast action API?

Adds actions to execute before (doFirst) or after (doLast) the main task action: tasks.named('test') { doFirst { println('Starting tests') } }. Multiple doFirst/doLast calls stack.

Q20. What is the difference between `tasks.named()` and `tasks.getByName()`?

`named()` returns a lazy `TaskProvider` — the task is not configured until accessed. `getByName()` returns the `Task` directly, eagerly triggering configuration. Use `named()` to avoid eager configuration.

Q21. What is a `TaskProvider`?

A lazy reference to a task that is only resolved when needed. Returned by `tasks.register()` and `tasks.named()`. Passing `TaskProviders` to `dependsOn` or `from()` avoids eager task configuration.

Q22. What is Gradle's incremental task API?

An API for tasks to process only changed inputs. A task with `@Incremental` input receives `InputChanges` with `addedFiles`, `modifiedFiles`, `removedFiles` — allowing partial reprocessing instead of full rebuild.

Q23. What is the `--scan` flag?

`./gradlew build --scan` — uploads a detailed build scan to `scans.gradle.com`. Shows timeline, task execution times, dependencies, test results, and performance analysis. Requires agreement on first use.

Q24. What is `./gradlew --profile`?

Generates an HTML build performance report in `build/reports/profile/`. Shows time spent in each phase and task without uploading to external services.

Q25. What does `./gradlew dependencies` show?

Prints the full dependency tree for all configurations. Use `--configuration` to filter: `./gradlew dependencies --configuration runtimeClasspath`.

Q26. What does `dependencyInsight` do?

`./gradlew dependencyInsight --dependency commons-logging` shows all paths in the dependency graph that lead to `commons-logging` and explains why its version was selected.

Q27. How do you run a single test class with Gradle?

`./gradlew test --tests 'com.example.MyServiceTest'`. For a single method: `--tests 'com.example.MyServiceTest.shouldReturnValidResult'`. Supports wildcards.

Q28. What is the Java toolchain in Gradle?

Allows specifying the exact JDK version to use for compilation and testing, independently of the JDK running Gradle. Gradle auto-provisions the correct JDK if not found: `java { toolchain { languageVersion = JavaLanguageVersion.of(17) } }`.

Q29. What is lazy configuration in Gradle?

Using `Provider` and `Property` instead of direct values. Values are computed only when needed. Enables proper task dependencies, up-to-date checking, and build cache compatibility.

Q30. What is a convention plugin in Gradle?

A plugin in `buildSrc` or an included build that encapsulates common configuration applied to multiple subprojects. Replaces `allprojects/subprojects` blocks with explicit, reusable configuration.

Q31. What is `buildSrc` in Gradle?

A special directory in the project root. Gradle compiles it first and makes its classes available on the `buildscript classpath`. Used to define convention plugins, custom tasks, and shared build logic.

Q32. What is the difference between `buildSrc` and an included build for plugins?

`buildSrc` is compiled per project and not cached between projects. Included builds (composite builds) can be shared across multiple root projects and participate in build caching.

Q33. What is a configuration cache in Gradle?

Caches the result of the Configuration phase. On subsequent builds with unchanged build scripts and inputs, Gradle skips Configuration entirely. Enabled with `--configuration-cache`.

Q34. What is the Gradle daemon?

A long-running background process that keeps the JVM warm between builds. Dramatically reduces build startup time. Enabled by default since Gradle 3.x. Run `./gradlew --stop` to kill all daemons.

Q35. What does `./gradlew --no-daemon` do?

Runs the build without the daemon — in a fresh JVM process. Used for CI where daemon process reuse is less beneficial and process isolation is preferred.